

# Smart Disaster Response MIS

Trigger Implementation & Use Cases

## DELIVERABLE — DATABASE TRIGGERS

Course: Database Systems

Campus: FAST-NUCES, Islamabad

Semester: Spring 2026

Document: Trigger Implementation

## Table of Contents

---

1. Overview & Justification
2. Trigger Inventory
3. Trigger Implementations
  - 3.1 Resource Dispatch — Inventory Control
  - 3.2 Team Assignment & Completion
  - 3.3 Patient Admission & Discharge
  - 3.4 Automated Audit Logging
  - 3.5 Low-Stock Alert
4. Triggers vs. Application Code

## 1. Overview & Justification

---

Database triggers enforce **non-bypassable business invariants** — rules that must hold regardless of whether the change originates from the API, a direct SQL session, or a concurrent transaction.

- **Inventory Control:** Prevents resource dispatches when stock is insufficient.
- **Hospital Capacity:** Decrements available beds on admission; rolls back if full.
- **Team Lifecycle:** Maintains team availability status across assignment and completion.
- **Automated Auditing:** Captures every INSERT and UPDATE on critical tables.
- **Low-Stock Alerting:** Notifies warehouse managers when inventory crosses the threshold.

## 2. Trigger Inventory

| Trigger Name                    | Table               | Event          | Category          |
|---------------------------------|---------------------|----------------|-------------------|
| trg_ResourceAllocation_Dispatch | ResourceAllocations | UPDATE         | Inventory         |
| trg_TeamAssignment_Insert       | TeamAssignments     | INSERT         | Team Lifecycle    |
| trg_TeamAssignment_Complete     | TeamAssignments     | UPDATE         | Team Lifecycle    |
| trg_Patient_Admission           | Patients            | INSERT         | Hospital Capacity |
| trg_Patient_Discharge           | Patients            | UPDATE         | Hospital Capacity |
| trg_FinanceTransaction_AuditLog | FinanceTransactions | INSERT         | Audit             |
| trg_EmergencyReport_AuditLog    | EmergencyReports    | INSERT, UPDATE | Audit             |
| trg_ResourceAllocation_AuditLog | ResourceAllocations | INSERT, UPDATE | Audit             |
| trg_Inventory_LowStockAlert     | WarehouseInventory  | UPDATE         | Alerting          |
| trg_ApprovalRequest_Execute     | ApprovalRequests    | UPDATE         | Workflow          |

Table 1 — Complete trigger inventory

## 3. Trigger Implementations

### 3.1 Resource Dispatch — Inventory Control

Fires **AFTER UPDATE** on **ResourceAllocations** when status transitions to '**dispatched**'. Deducts stock atomically and raises an error triggering ROLLBACK if stock would go negative.

```
CREATE OR ALTER TRIGGER trg_ResourceAllocation_Dispatch
ON ResourceAllocations
AFTER UPDATE
AS
BEGIN
SET NOCOUNT ON;
IF UPDATE(status)
BEGIN
UPDATE wi
SET wi.quantity_available =
wi.quantity_available - i.quantity_allocated
FROM WarehouseInventory wi
JOIN inserted i ON i.resource_id = wi.resource_id
AND i.warehouse_id = wi.warehouse_id
JOIN deleted d ON d.allocation_id = i.allocation_id
WHERE i.status = 'dispatched'
AND d.status != 'dispatched';

IF EXISTS (SELECT 1 FROM WarehouseInventory
WHERE quantity_available < 0)
BEGIN
RAISERROR('Insufficient stock.', 16, 1);
ROLLBACK TRANSACTION;
END
END
END;
```

*Listing 1 — trg\_ResourceAllocation\_Dispatch*

### 3.2 Team Assignment & Completion

```
-- On new assignment
CREATE OR ALTER TRIGGER trg_TeamAssignment_Insert
ON TeamAssignments AFTER INSERT AS
BEGIN
SET NOCOUNT ON;
UPDATE RescueTeams SET status = 'assigned'
WHERE team_id IN (SELECT team_id FROM inserted);

INSERT INTO DispatchLogs (team_id, log_type, log_message, logged_at)
SELECT team_id, 'ASSIGNED',
'Team assigned to report ' + CAST(report_id AS VARCHAR),
GETDATE()
FROM inserted;
END;
```

```
-- On mission completion
CREATE OR ALTER TRIGGER trg_TeamAssignment_Complete
ON TeamAssignments AFTER UPDATE AS
BEGIN
SET NOCOUNT ON;
IF UPDATE(status)
UPDATE RescueTeams SET status = 'available'
WHERE team_id IN (
SELECT team_id FROM inserted WHERE status = 'completed');
END;
```

*Listing 2 — Team lifecycle triggers*

### 3.3 Patient Admission & Discharge

```
CREATE OR ALTER TRIGGER trg_Patient_Admission
ON Patients AFTER INSERT AS
BEGIN
SET NOCOUNT ON;
UPDATE Hospitals SET available_beds = available_beds - 1
WHERE hospital_id IN (SELECT hospital_id FROM inserted);

IF EXISTS (SELECT 1 FROM Hospitals WHERE available_beds < 0)
BEGIN
RAISERROR('No beds available.', 16, 1);
ROLLBACK TRANSACTION;
END
END;

CREATE OR ALTER TRIGGER trg_Patient_Discharge
ON Patients AFTER UPDATE AS
BEGIN
SET NOCOUNT ON;
IF UPDATE(condition)
UPDATE Hospitals SET available_beds = available_beds + 1
WHERE hospital_id IN (
SELECT hospital_id FROM inserted
WHERE condition IN ('discharged', 'deceased'));
END;
```

*Listing 3 — Patient admission and discharge triggers*

### 3.4 Automated Audit Logging

```
CREATE OR ALTER TRIGGER trg_EmergencyReport_AuditLog
ON EmergencyReports AFTER INSERT, UPDATE AS
BEGIN
SET NOCOUNT ON;
INSERT INTO AuditLog
(table_affected, action_type, record_id,
changed_by, action_timestamp, notes)
SELECT 'EmergencyReports',
CASE WHEN EXISTS (SELECT 1 FROM deleted)
THEN 'UPDATE' ELSE 'INSERT' END,
i.report_id,
i.operator_id,
GETDATE(),
'Status: ' + ISNULL(d.status, 'NEW') + ' -> ' + i.status
FROM inserted i
LEFT JOIN deleted d ON d.report_id = i.report_id;
END;
```

*Listing 4 — EmergencyReports audit trigger*

### 3.5 Low-Stock Alert

```
CREATE OR ALTER TRIGGER trg_Inventory_LowStockAlert
ON WarehouseInventory AFTER UPDATE AS
BEGIN
SET NOCOUNT ON;
INSERT INTO SystemNotifications
(notification_type, message, created_at, target_role)
SELECT 'LOW_STOCK',
r.resource_name + ' in ' + w.name + ' is below threshold (' +
CAST(i.quantity_available AS VARCHAR) + ' remaining).',
GETDATE(), 'warehouse_manager'
FROM inserted i
JOIN Resources r ON r.resource_id = i.resource_id
JOIN Warehouses w ON w.warehouse_id = i.warehouse_id
WHERE i.quantity_available < i.threshold_level
AND (SELECT quantity_available FROM deleted
WHERE resource_id = i.resource_id
AND warehouse_id = i.warehouse_id) >= i.threshold_level;
END;
```

*Listing 5 — Low-stock alert trigger*

## 4. Triggers vs. Application Code

| Concern             | Application Code                                 | Database Trigger                                |
|---------------------|--------------------------------------------------|-------------------------------------------------|
| Bypass resistance   | Can be bypassed by direct SQL access or API bugs | Fires for all DML regardless of source          |
| Concurrency safety  | Race conditions possible between check and write | Executes within transaction — no race window    |
| Audit completeness  | Depends on every code path calling the logger    | Fires unconditionally on every relevant event   |
| Deployment coupling | Rule coupled to application release cycle        | Lives at database tier; survives app re-deploys |

Table 2 — Triggers vs. application code comparison